Category:

Reversing

Name:

Flag Encoder

Message:

The restored executable turned out to be a flag encoder. We also found an encoded flag string as indicated below. This string was generated by this flag encoder based on the correct flag. Can you recover the flag?

Encoded flag string: lBaY4gCyygnZ3YFweaCbCfnGm1JcSiNpzE8mjZGCcRA=

Repaired File in previous challenge "corrupted": repaired.zip

Instructions:

1. As displayed after the execution, give an argument to the exe. You will see the encoded string as an output. This exe gets the given argument and provides the encoded string as output. The task is to understand the encoding process to reverse the provided encoded flag to get the flag.

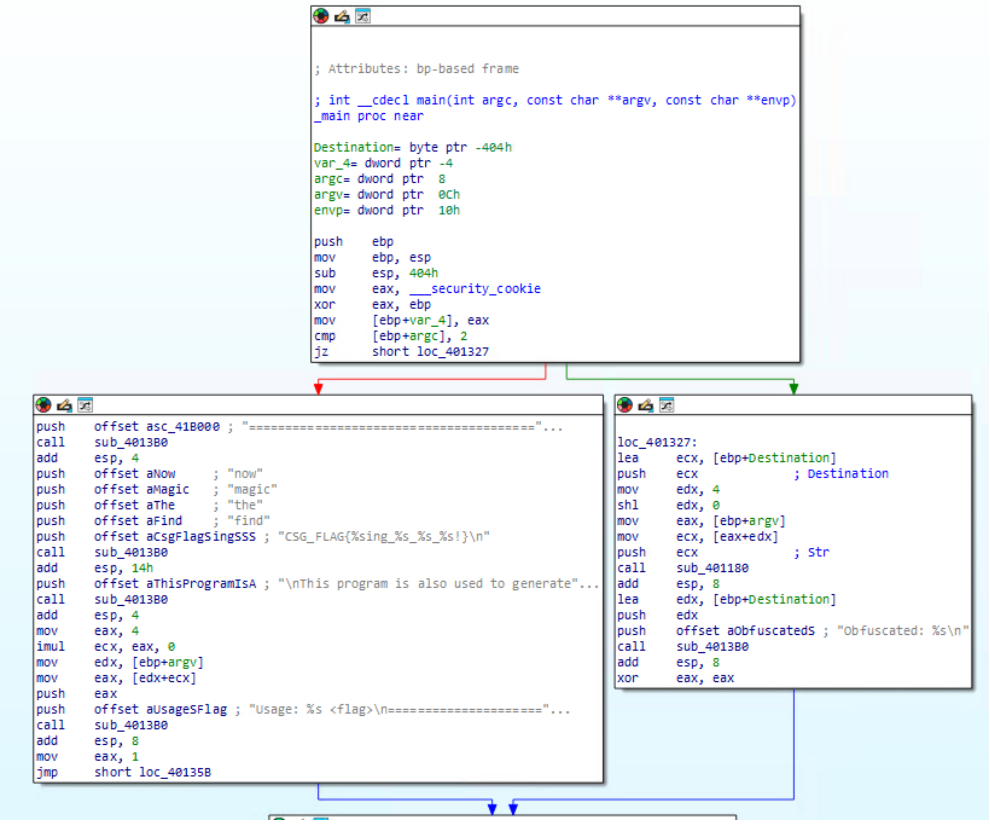
```
C:¥Users¥yorune¥Documents¥2025AJCCBC>repaired.exe Testtest
Obfuscated: Tv0XTu4DNNR=
```

2. Check **repaired.exe** using IDA Free or any other tools. Pay attention to the characteristic strings among Strings, which looks like a custom Base64 table. This indicates that Base64 could be included in the encoding process.

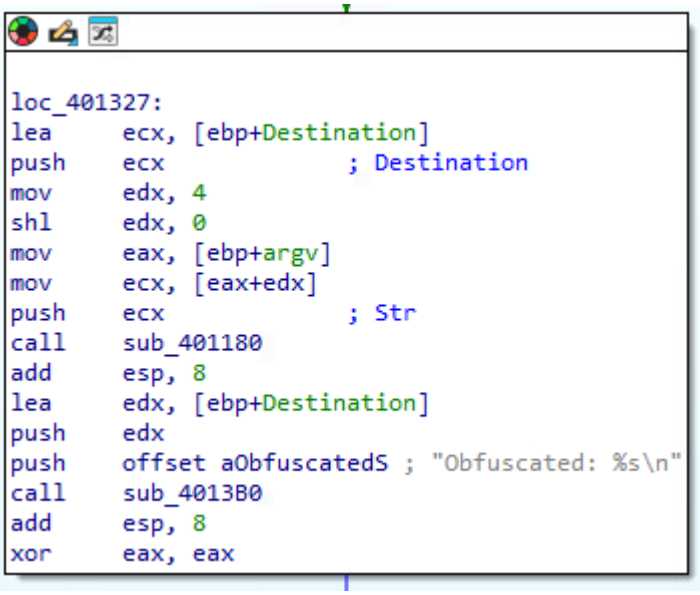
IDA View-A | Strings | Hex View-1

Address	Length	Type	String
.rdata:0041...	00000011	C	7qrnsZ6fbEYa5ezF
.rdata:0041...	00000011	C	Rw8eT0aLIyzLTewC
.rdata:0041...	00000042	C	Z3br47h1nksMeacdfgijlmopqtuvwxyzABCDEFGHIJKLNOPQRSTUVWXY025689+/=
.rdata:0041...	00000042	C	P4ndaBi7esF0oDbcfghjklmpqrtuvwxyzACEGHIJKLMNOQRSTUVWXYZ1235689+/=
.rdata:0041...	00000042	C	W0lf57r34tHOneyabcdghijkmopqsuvwxzABCDEFGIJKLMNPQRSTUVXYZ12689+/=
.rdata:0041...	00000042	C	M0nk3yBetsFrui7EabcdfghjlmopqvwxzACDGHIJKLNOPQRSTUVWXYZ1245689+/=
.rdata:0041...	00000042	C	F0x3sDr1nkJuiceabdfghjlmopqtvwyzABCEGHIKLMNOPQRSTUVWXYZ2456789+/=
.rdata:0041...	00000042	C	D09esToOp3nM47habcdfgijklmqrtuvwxyzABCEFGHIJKLNPQRSUVWXYZ12568+/=
.rdata:0041...	00000041	C	atS4vesTh3Worldbcfgijkmnpquwxyz ABDEFGHIJKLMNOPQRUVXYZ01256789+/=

3. By opening the Graph View, we observe that the main function branches based on the number of arguments passed at runtime. If only one argument is provided, the execution proceeds to the lower-left block, which displays a usage message. If two arguments are given, the flow continues to the lower-right block, where the input flag is obfuscated. Therefore, it is likely that the encoding algorithm is implemented within the lower-right block.

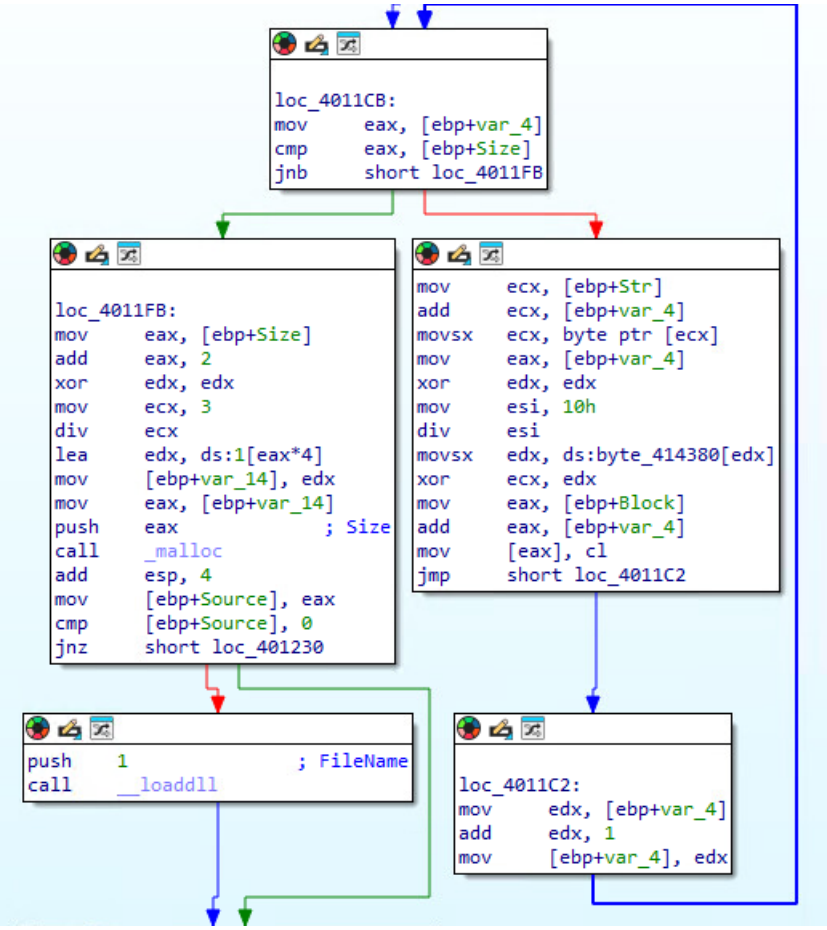


4. In the lower-right block, after receiving the argument (argv), a function call to sub_401180 occurs before the encoded string is displayed using the format "**Obfuscated: %s¥n**". This suggests that the encoding process is handled within the sub_401180 function. Double-clicking this function will allow further analysis.

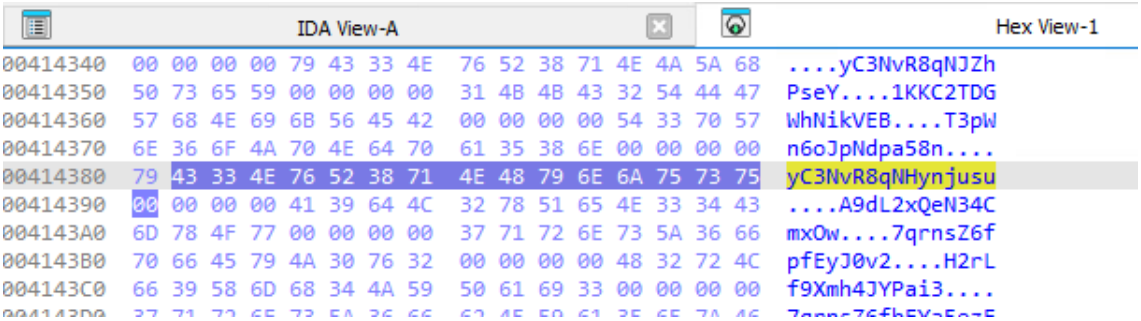


5. After a conditional branch, we see that the middle-right block processes the input string through

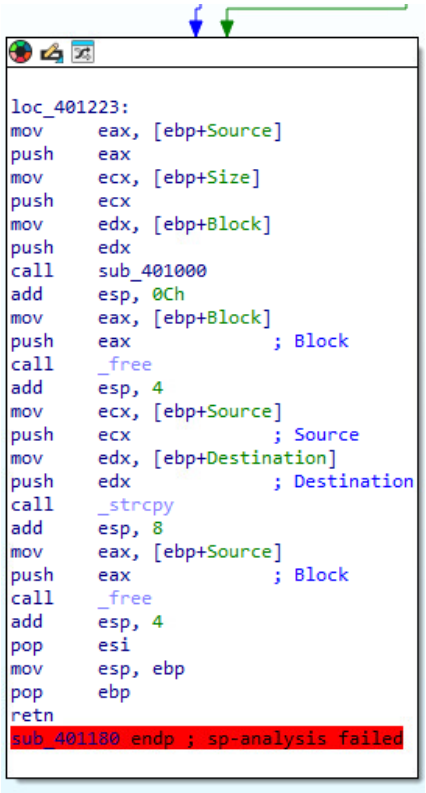
a loop structure, indicating iterative manipulation of the input.



6. In the right block, an XOR operation is performed using a key stored at **ds:byte_414380**. By double-clicking **byte_414380** and examining its value in the Hex View, we find that it contains the string " **yC3NvR8qNHynjusu**", which serves as the XOR key.

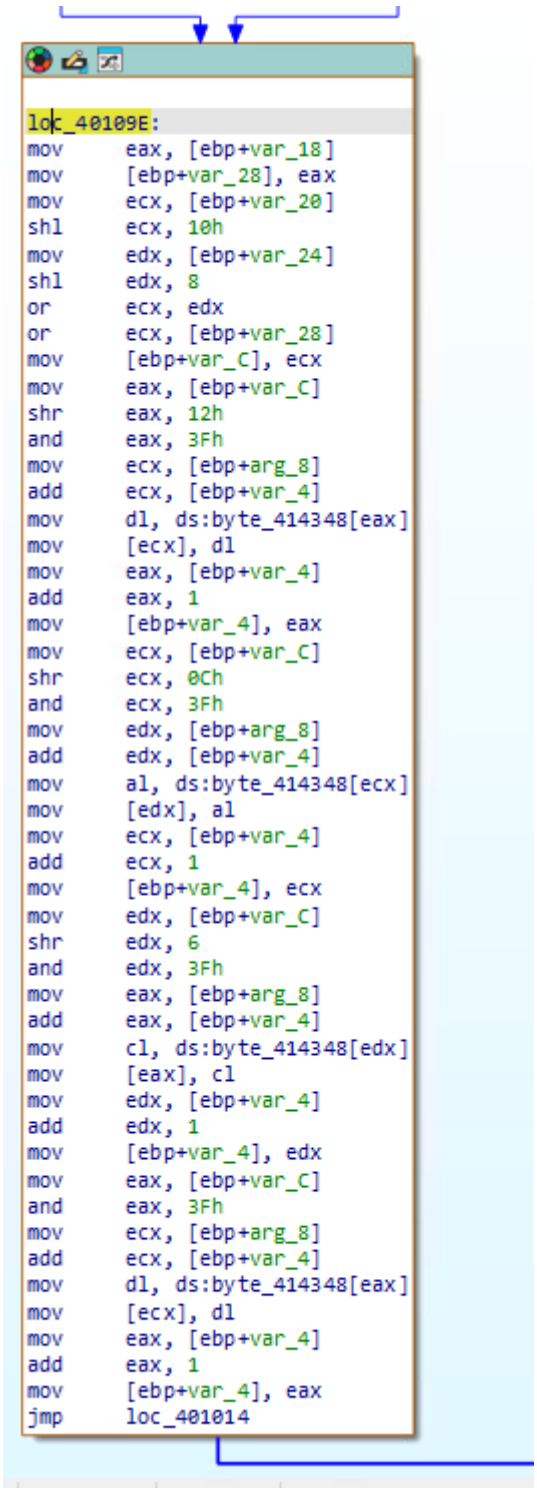


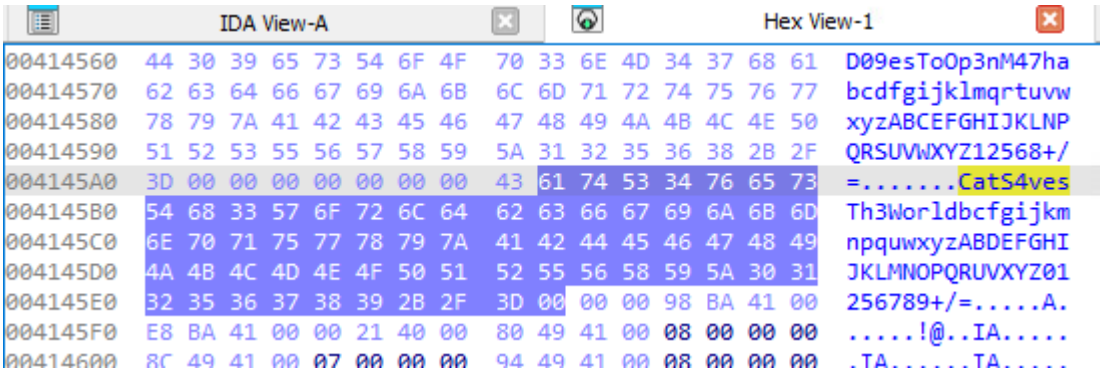
7. Once the operations in the right block are completed, the execution moves to the left block, where additional processing is performed. Within this block, a new function, sub_401000, is called. Double-clicking on this function reveals its internal behavior.



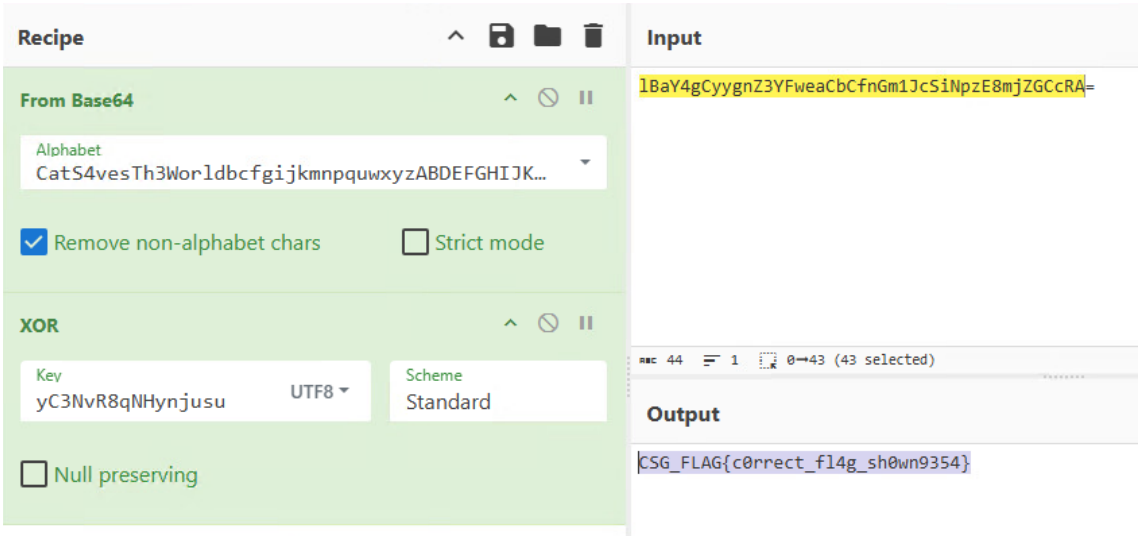
```
loc_401223:
mov     eax, [ebp+Source]
push    eax
mov     ecx, [ebp+Size]
push    ecx
mov     edx, [ebp+Block]
push    edx
call    sub_401000
add     esp, 0Ch
mov     eax, [ebp+Block]
push    eax             ; Block
call    _free
add     esp, 4
mov     ecx, [ebp+Source]
push    ecx             ; Source
mov     edx, [ebp+Destination]
push    edx             ; Destination
call    _strcpy
add     esp, 8
mov     eax, [ebp+Source]
push    eax             ; Block
call    _free
add     esp, 4
pop     esi
mov     esp, ebp
pop     ebp
retn
sub_401180 endp ; sp-analysis failed
```

8. Within sub_401000, bitwise operations typical of Base64 encoding are present, such as **mov + shl + or**, **and + 0x3F**, **mov + dl** etc. The Base64 encoding is performed by using the table stored in **byte_414348.** By double-clicking **byte_414348** and examining its value in the Hex View, we find that it contains the custom Base64 table "**CatS4vesTh3Worldbcfgijkmnpquwxyz**ABDEFGHIJKLMNOPQRUVXYZ01256789+/=".


loc_40109E:
mov eax, [ebp+var_18]
mov [ebp+var_28], eax
mov ecx, [ebp+var_20]
shl ecx, 10h
mov edx, [ebp+var_24]
shl edx, 8
or ecx, edx
or ecx, [ebp+var_28]
mov [ebp+var_C], ecx
mov eax, [ebp+var_C]
shr eax, 12h
and eax, 3Fh
mov ecx, [ebp+arg_8]
add ecx, [ebp+var_4]
mov dl, ds:byte_414348[eax]
mov [ecx], dl
mov eax, [ebp+var_4]
add eax, 1
mov [ebp+var_4], eax
mov ecx, [ebp+var_C]
shr ecx, 0Ch
and ecx, 3Fh
mov edx, [ebp+arg_8]
add edx, [ebp+var_4]
mov al, ds:byte_414348[ecx]
mov [edx], al
mov ecx, [ebp+var_4]
add ecx, 1
mov [ebp+var_4], ecx
mov edx, [ebp+var_C]
shr edx, 6
and edx, 3Fh
mov eax, [ebp+arg_8]
add eax, [ebp+var_4]
mov cl, ds:byte_414348[edx]
mov [eax], cl
mov edx, [ebp+var_4]
add edx, 1
mov [ebp+var_4], edx
mov eax, [ebp+var_C]
and eax, 3Fh
mov ecx, [ebp+arg_8]
add ecx, [ebp+var_4]
mov dl, ds:byte_414348[eax]
mov [ecx], dl
mov eax, [ebp+var_4]
add eax, 1
mov [ebp+var_4], eax
jmp loc_401014



9. Based on the above analysis, this program takes an input string, applies an XOR operation using the key " **yC3NvR8qNHynjusu**", and then converts the result into a custom Base64-encoded string (**CatS4vesTh3WorldbcfgijkmnpquwxyzABDEFGHIJKLMNOPQRUVXYZ01256789+/=**) to obfuscate the input. Given the provided obfuscated output string (**lBaY4gCyygnZ3YFweaCbCfnGm1JcSiNpzE8mjZGCcRA=**), reversing this process—i.e., Base64 decoding followed by XOR decryption—will reveal the original flag.



Flag is:

CSG_FLAG{c0rrect_fl4g_sh0wn9354}